

CUBE NANOSATELLITE POSITIONAL FUNCTIONALITY USING DRONE SWARM SIMULATION

Skyler Do

Department of Industrial and Systems Engineering, Graduate School of Engineering, University of Michigan Ann Arbor
skylerdo@umich.edu

Supervisors: Shigeru Sunada, Takaya Inamori

Graduate School of Engineering, Nagoya University
shigeru.sunada@mae.nagoya-u.ac.jp
inamori@nuae.nagoya-u.ac.jp

ABSTRACT

This project was a collaboration between two labs, one in Aerospace (headed by Professor Sunada), and one in Space Systems (headed by Professor Inamori). The objective of this collaboration is to help validate the Inamori lab's cube nanosatellite's swarm positional functionality ahead of its scheduled 2028 launch on a JAXA shuttle.

1. INTRODUCTION

The MAGNARO II (Magnetically separating Nano-satellite with Rotation for Orbit control) nano satellite project is a small, 10cm cubed device that is designed to orbit the earth and send telemetry and images back to a ground station. The unit will be deployed as a docked pair, before separating into two individual units that will fly in formation with one another. The plan is to deploy multiple pairs of these units together, ultimately creating a "swarm" of cube satellites that will be able to relay information, take pictures, and send information back to ground stations.



Figure 1, the MAGNARO in its docked configuration

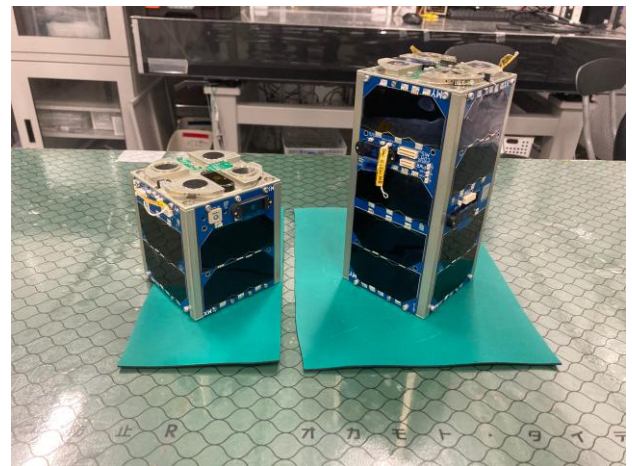


Figure 2, the separated MAGNARO units

By using its built in magnetorquer actuator, the MAGNARO is able to orient itself in space as it orbits the earth. By carefully controlling the MAGNARO orientation, it is possible to indirectly control the air forces acting on the unit. Using these air forces, the unit is thus able to control its trajectory.

Extending upon this functionality, it would be possible for a swarm of these units to be able to fly in formation with one another. Validating that functionality is the key goal of this project. To help with this, the aerospace engineering experts of the Sunada lab have been brought together. The goal for this project was to begin the process of validating the MAGNARO's swarm positional functionality. The initial plan is to have a number of drones carrying the cubesats, moving them to different positions and orientations. This is to simulate the high altitude formation flying of the MAGNARO units.

2. RESEARCH AND DEVELOPMENT

2.1 DUAL VALIDATION MODEL

In order to avoid disrupting the ongoing development efforts of the MAGNARO, which is undergoing HILS (Hardware-in-the-loop) testing, the decision was made to pursue a dual validation process. Both validation methods would be independent of one another, with each signal path between agents in the system being called out and elaborated on in table 1. The full dual validation system context diagram can be seen in figure 3.

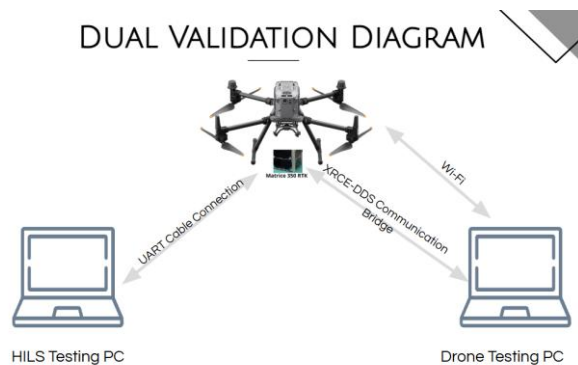


Figure 3, Dual validation diagram

Table 1, Validation Plan

Rx/Tx Path	HILS Testing	Drone Testing
PC to Cubesat	Environment Data, Orbital Information	Swarm formation commands
Cubesat to PC	Telemetry, MTQ Command Info	Formation status, MTQ Commands
PC to Drone	NA	Positional Command (Transformed MTQ Command)
Drone to PC	NA	Position Status, Drone status

As the project developed and more things became clear, we pivoted based on the needs of the project. I will elaborate on a few of the biggest decisions. In addition, there were no available drones to use, and only one MAGNARO unit that was preoccupied. Therefore, the decision was made for this entire project to be a virtual simulation. My task then became to understand what software and tools were available for this purpose.

2.2 DJI DRONES

Professor Nishio had some suggestions about using DJI drones for this project, but we quickly ran into the roadblock that DJI drone software is proprietary and copyright projected. There is a DJI Software Development Kit, but I did not find many community resources around integration with other tools and simulation environments.

2.3 MAVLINK

Professor Miwa from Tokyo also suggested I look into the MAVLink Protocol, a communication protocol used by MAVROS for integration with ROS. However upon further research, the latest communication protocol for ROS2 and PX4, is XRCE-DDS Agent. This alongside the fact that the weaker processor on the MAGNARO is unable to support a full ROS installation (it required Micro ROS) drove the decision to adopt XRCE-DDS over MAVLink.

2.4 FINAL TOOLS LIST

After much exhaustive research, this was the optimal selection of tools I compiled for this project.

Table 2, Software Collection and explanations

Software	Function
Gazebo	Physics engine, simulation environment
PX4	Open source drone software stack. Handles low level hardware control.
ROS2	An open source robotics communication middleware that allows multiple software components in different parts of the robotics system to talk to one another. Abstracts a lot of functionality into useful libraries common to robotics development.
Micro ROS	Lightweight version of ROS2. Requires less computing resources, but is similarly less capable than full ROS. Useful for particular nodes in a robotics system that are resource constrained and don't need full ROS functionality.
Linux	Open source OS necessary for all these software tools
XRCE-DDS Agent	Modern communication protocol that exposes PX4 internal topics and variables to ROS. Useful for viewing and modifying PX4 behavior remotely.
QGroundControl	Ground station control software. Used to plan flight paths, command

	direction, view camera feed (if available) and change command mode. For a drone with Gimbal, this software also commands the gimbal.
--	--

3. SIMULATIONS AND DEMONSTRATIONS

I was able to achieve some basic functionality with the tools that I researched, and made good first steps towards the eventual goal of a fully realized simulation environment. And keeping in mind the eventual goal of physical demonstrations using real drones and real MAGNARO units, I determined that this simulation would still be a valuable pursuit since the software can be re-used on real drones and ground control PCs once they have been purchased. The only part that will be unused is the simulation environment. All software and development done here will eventually be useful in other parts of the real system. Table 2 below shows the allocation of software to hardware components.

Table 2, Software Allocation to Hardware

Software	Magnaro	Drone	Ground Station
Gazebo	×	×	×
PX4	×	✓	×
ROS	×	✓	✓
Micro ROS	✓	×	×
Linux	×	×	✓
XRCE-DDS Agent	×	✓	✓
QGroundControl	×	×	✓

In addition, I was able to capture some of the functionality of my simulations. All simulation demo videos can be viewed at: [DEMONSTRATION VIDEOS](#)

3.1 TOOL INTEGRATION AND BASIC ENVIRONMENT

I was able to get the various tools and software components to talk together in order to achieve a basic drone simulation. I will explain the various programs and processes in each window.

Top left: Command terminal for QGroundControl.

Bottom left: PX4 flight stack. Simulation of the software on the drone.

Center: Gazebo, the simulation environment where everything “happens” You can see the spawned drone hovering.

Right: QGroundControl, the ground control software that links to PX4.

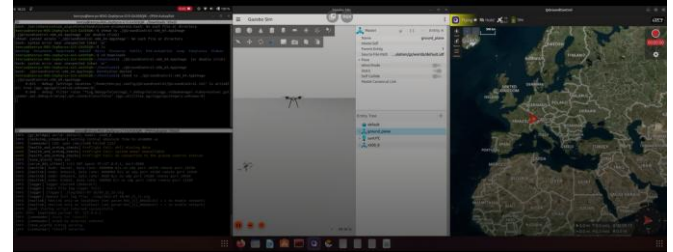


Figure 4, First flight

Commanding the drone to fly in the bottom left terminal makes it fly in the Gazebo simulator.

3.2 MULTI DRONE SIMULATION

Progressing from a single drone to a multi drone simulation was deceptively difficult. There were naming and conflict errors with multiple instances of the flight stack, and navigating the environmental variables to adjust was quite the headache. After much trial and error, I was able to spawn 3 separate instances of the PX4 flight stack, each with its accompanying drone. With the exception of the bottom left terminal (which is the QGroundControl terminal), the other 3 terminals are the PX4 stacks.

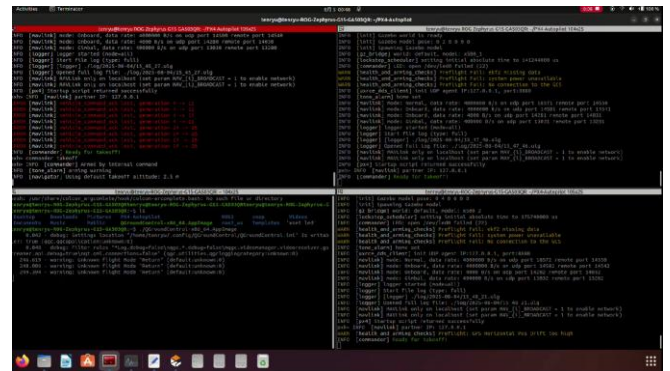


Figure 5, flight stack for triple drone simulation

You can see the multiple drones spawned in the figure below.

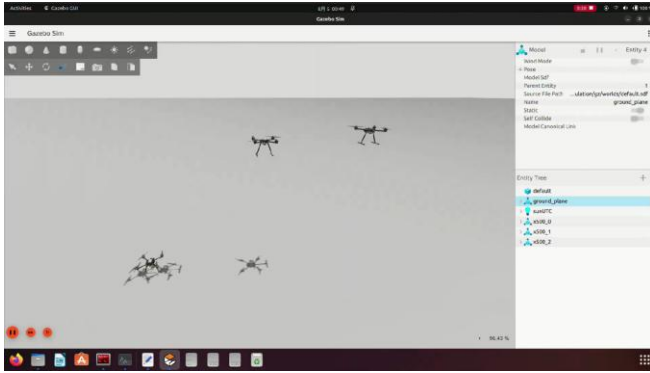


Figure 6, 3 drones simulated in the Gazebo environment

3.3 SINGLE DRONE TELEOPERATION

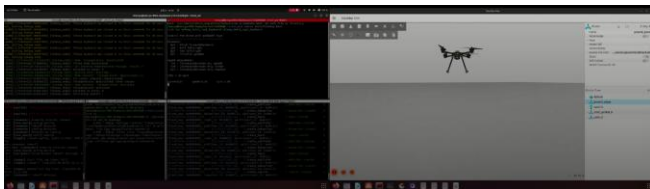


Figure 7, Single drone Teleoperation setup

In the figure above, I was able to achieve keyboard controlled teleoperation of a single drone. This demonstrates integration of all the software components laid out in table 2. Using QGroundControl, I activate a custom control mode that is looking for inputs. A ROS node is activated and reads my keyboard inputs, another sends that to the XRCE-DDS Agent, which then passes that to the PX4 software. That control software is then reflected in the simulated control of the drone. I am able to move the drone in different directions, tilt it, and have it orbit in place, and raise and lower the altitude.

3.4 SINGLE DRONE GIMBAL CONTROL

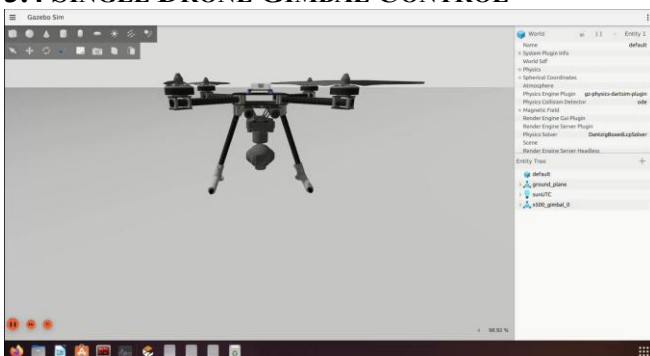


Figure 8, Single Drone Gimbal Simulation

In addition, I was able to simulate a drone with a gimbal and control the gimbal orientation using QGroundControl. Despite the simulation showing a camera, in reality this would be a MAGNARO unit. They are roughly the same size and weight, and suit our objective of “orient the MAGNARO in 3D space” perfectly.

4. FUTURE WORK

This report only chronicles the beginning of the Drone-Satellite project. In this section I will briefly discuss my thoughts about the future work of this project, and the necessary steps. While there was good progress on the simulation front, a number of action items remain. In addition, further collaboration and support from other members of the participating laboratories will be required.

4.1 FUTURE SIMULATION WORK

First, I would like to complete the teleoperation of multiple drone units at once. Ideally, keyboard input could be used to command preset formations. The individual pieces for this functionality already exist: reading keyboard input, multi drone simulation, and commanding location using QGroundControl.

Second, full manual control over the gimbal orientation. Currently, there are settings in QGroundControl to adjust the gimbal orientation in 90 degree increments using the GUI, but not yet a way to directly modify internal gimbal orientation variables using the ROS - PX4 bridge I developed for the teleoperation functionality.

Third, MAGNARO and PC communication integration. MAGNARO needs an installation of micro ROS to participate in this system. The practical implementation of this has yet to be explored.

4.2 INAMORI LAB FUTURE WORK

MAGNARO and PC communication integration. I will need the Inamori lab’s help with understanding the software stack on the MAGNARO to understand how we should go about installing micro ROS without disturbing any existing software and functionality, as well as how to pass information and commands from the MAGNARO to the micro ROS software. ROS is also a network based protocol and does not require a wired connection between all agents in the system to communicate, but it is worth exploring if communication can occur at the desired distances and using the existing MAGNARO radio communications.

4.3 SUNADA LAB FUTURE WORK

The model for transforming a magnetorquer command into a real orientation command for the drone could be an entire research project unto itself. The lab needs to understand the high altitude space environment the MAGNARO is expected to operate in, investigate its aerodynamics, understand the expected torque values from the magnetorquer interactions with the Earth’s magnetic field, and how that will ultimately translate into an appropriate position and orientation command for the drone. Two models will need to be developed, one for the cube shaped MAGNARO unit, and one for the larger rectangular MAGNARO unit.

4.4 REAL DEMONSTRATIONS AND TESTING

Finally of course, all of this needs to be deployed on real hardware. For the swarm functionality, multiple MAGNARO units and drones will be required. Adapting the

physical gimbal to interface with the MAGNARO. Depending on the specific gimbal selected, this could be as simple as screwing in the MAGNARO using the camera interface, or 3D printing some new interface for the connection.

Simulations rarely cover *all* of the potential use cases and failure modes of a system under testing, and this case is no different. Practical issues such as wind, network connection strength, battery, and weight concerns are all simplified and/or omitted in simulations. These will have to be kept in mind for the future.

5. CONCLUSIONS

As I thought about the best way to accomplish the project objectives, I realized that the decision to use drones to carry the nanosatellites was made before I began work on the project. In retrospect, I think the project team needs to better define the functionality they are attempting to test. If the objective is long range high altitude communications, there is no need to use technically demanding drones. A simple communication test from the base of a tall building to its peak would be sufficient.

More fundamentally, even operating with the assumption that we are able to develop a drone swarm system that can position nanosatellites in the desired orientations and positions, *that does not prove* that they will operate as desired in the space environment that they will be deployed. This is doubly so for the magnetorquer to orientation model that will be crucial for controlling the MAGNARO trajectory.

However, I still think the progress that I've made here will be valuable to the lab, and I have learned a great deal about drones, software development, and the various tools used to achieve my demonstrations.

6. ACKNOWLEDGEMENTS

My deepest gratitude to the following people who supported me in this project, both professionally and personally:

Professor Shigeru Sunada
Professor Takaya Inamori
TA: Yoshida Kodai

7. REFERENCES

[1] Aerospace Vehicle Dynamics Research Group, "Nanosat Research," Nagoya University, accessed in 2025, https://nanosat.nuae.nagoya-u.ac.jp/research_nanosat_en.html.

[2] MAVLink, accessed in 2025, <https://mavlink.io/>.

[3] ArduPilot: Copter, accessed in 2025, <https://ardupilot.org/copter/>.

[4] Qiita — srs, "How to setup PX4 SITL with ROS2 and XRCE-DDS Gazebo simulation on Ubuntu 22," accessed in 2025, <https://qiita.com/srs/items/1b14bdf0f1fd3c49cf49>.

[5] ArduPilot Documentation, accessed in 2025, <https://ardupilot.org/ardupilot/index.html>.

[6] DJI Payload SDK Tutorial, accessed in 2025, <https://developer.dji.com/doc/payload-sdk-tutorial/en/>.

[7] DJI Enterprise — Matrice 350 RTK (Japan), accessed in 2025, <https://enterprise.dji.com/jp/matrice-350-rtk>.

[8] DJI SDK — Payload-SDK GitHub repository, accessed in 2025, <https://github.com/dji-sdk/Payload-SDK>.

[9] NASA — Appendix I: Verification and Validation Plan Outline, accessed in 2025, <https://www.nasa.gov/reference/appendix-i-verification-and-validation-plan-outline/>.

[10] YouTube video "u4LM1R8SLbA" (title not provided), accessed in 2025, <https://www.youtube.com/watch?v=u4LM1R8SLbA&t=27s>.

[11] PX4 Docs — Simulation, accessed in 2025, <https://docs.px4.io/main/en/simulation/>.

[12] PX4 Docs — ROS 2 User Guide, accessed in 2025, https://docs.px4.io/main/en/ros2/user_guide.html.

[13] Kuat-Telegenov, "How to setup PX4 SITL with ROS2 and XRCE-DDS Gazebo simulation on Ubuntu 22," Notion page, accessed in 2025, <https://kuat-telegenov.notion.site/How-to-setup-PX4-SITL-with-ROS2-and-XRCE-DDS-Gazebo-simulation-on-Ubuntu-22-e963004b701a4fb2a133245d96c4a247>.